VISIONGUARD

AI-generated vs real image classification

Task 1: Data Understanding and Versioning
Task 2: Model Development with TensorFlow/Keras

Submission detail	Value
Name	Hayyan Haider
Roll number	23L-0727
Project	23L-0727_A2_Ops
Date	6 November 2026 (Asia/Karachi)
Graded source commit	248e204c3b5fbf3978a09c94a4b69b679cdc2104
GitHub	https://github.com/Muzxa/23L-0727_A2_Ops
DagsHub	https://dagshub.com/Muzxa/23L-0727_A2_Ops
Colab notebook	https://colab.research.google.com/drive/1DUeUYKEAaNH7_sl6O8K48r6MESux_Htl
Scope	Tasks 1 and 2 only. Deployment/API work belongs to later tasks.

AI assistance disclosure. OpenAI Codex helped prepare and repair the audit and training code, operated the Colab experiment, inspected evidence, and drafted this report. The reported measurements come from the actual assignment dataset and saved experiment outputs. The student should review the code, results, and interpretations before submission.

The experiment uses folder labels (AI=1, real=0). These labels are dataset annotations; a model score is not proof of image authenticity. All test figures reuse one inference pass per final model.

Task 1 - Data Understanding and Versioning

1.1 - Environment and GPU

The local project uses an isolated Python virtual environment, separating its dependencies from system Python. The graded model experiment ran in a separate Google Colab runtime with an NVIDIA T4 GPU. TensorFlow GPU detection and the runtime/package record are shown below; exact installed package versions are supplied in requirements-lock.txt.

Setting	Observed value
Python	3.13.16 (main, Oct 1 2026, 15:40:24) [GCC 13.3.0]
TensorFlow / Keras	2.21.0 / 3.13.2
GPU devices	/physical_device:GPU:0
CUDA / cuDNN	12.5.1 / 9
Random seed / image size / batch	42 / 224 x 224 RGB / 32

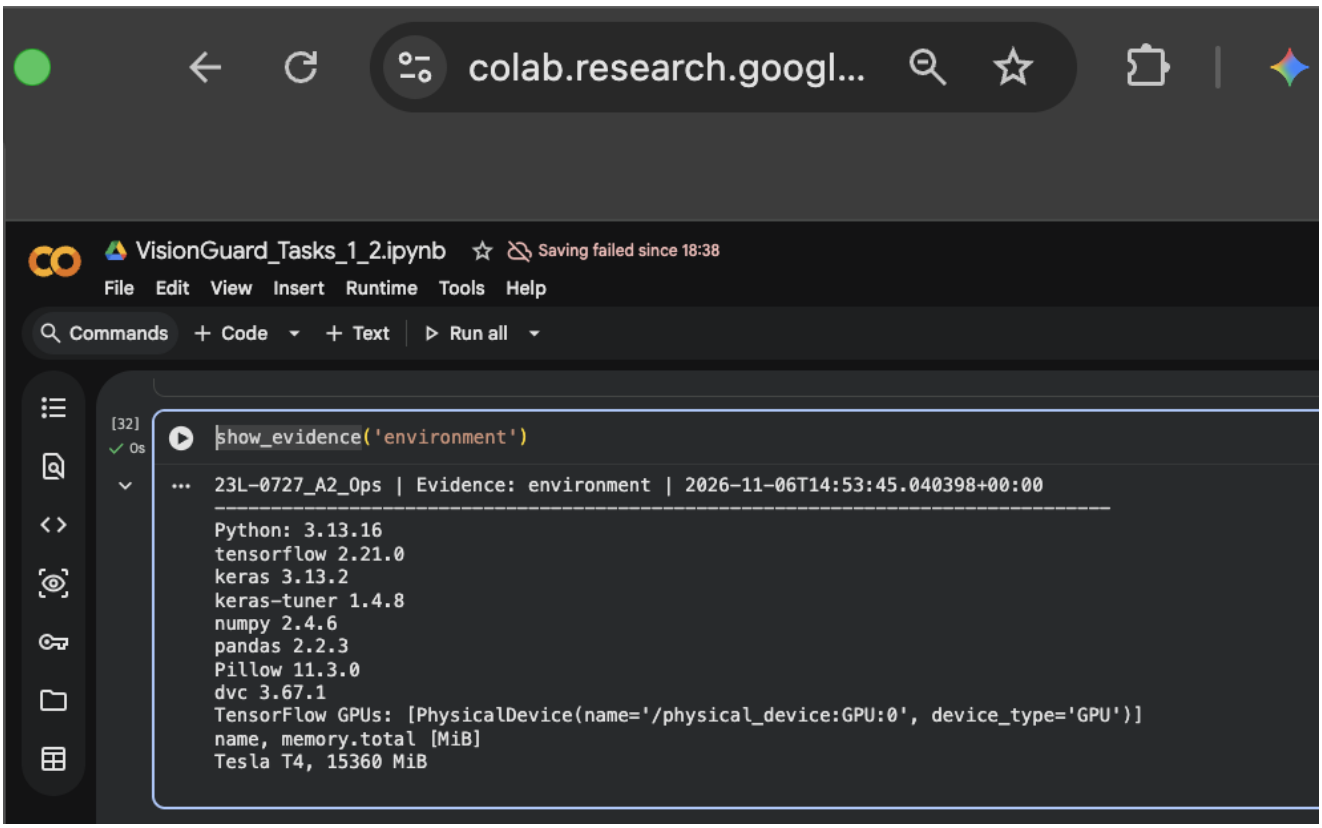


Figure 1. Actual Colab package versions and GPU evidence, with project context.

Commands and choices. python -m venv creates isolation; pip installs the declared dependencies. TensorFlow list_physical_devices checks GPU visibility; pip freeze records exact versions. Keeping Colab's supplied TensorFlow/CUDA combination avoids replacing its working GPU stack.

Observed issue. Google Drive was full, preventing notebook saves. Training checkpoints and outputs were therefore written under /content/23L-0727_evidence and downloaded to the local submission bundle. Runtime-local files alone are not persistent backups.

1.2 - Original dataset audit

The script recursively walks the original Kaggle files. For each candidate image it records class, file size, width, height, colour mode, format, byte SHA-256, decoded RGB pixel SHA-256, and any decoding error. Pillow verify plus a full decode checks readability. The nearest recognized parent folder assigns the label: AiArtData is AI and RealArt is real.

Class	Files	Readable	Width min/median/max	Height min/median/max
AI	538	538	225 / 1028.5 / 6000	135 / 800 / 4001
Real	435	435	183 / 940 / 8495	158 / 720 / 7091

Class	File bytes: minimum	Median	Maximum
AI	11,738	199,802.5	5,626,183
Real	11,311	144,982	10,195,352

There are 973 readable images and no unreadable candidates. One duplicate group contributes one extra file; decoded-pixel deduplication also removes one extra image. No cross-label pixel conflicts were found. AI images account for 55.3% of the original collection, so accuracy should be considered alongside precision, recall, F1, and ROC-AUC.

Split	Real	AI	Total
Train	304	376	680
Validation	65	81	146
Test	66	80	146

Deduplication happens before a seeded, stratified 70/15/15 split. Split lists retain original paths and hashes, making membership auditable. The retained pixel-hash sets are disjoint across train/validation/test. This only detects identical images: near duplicates and shared web sources may still leak visual information.

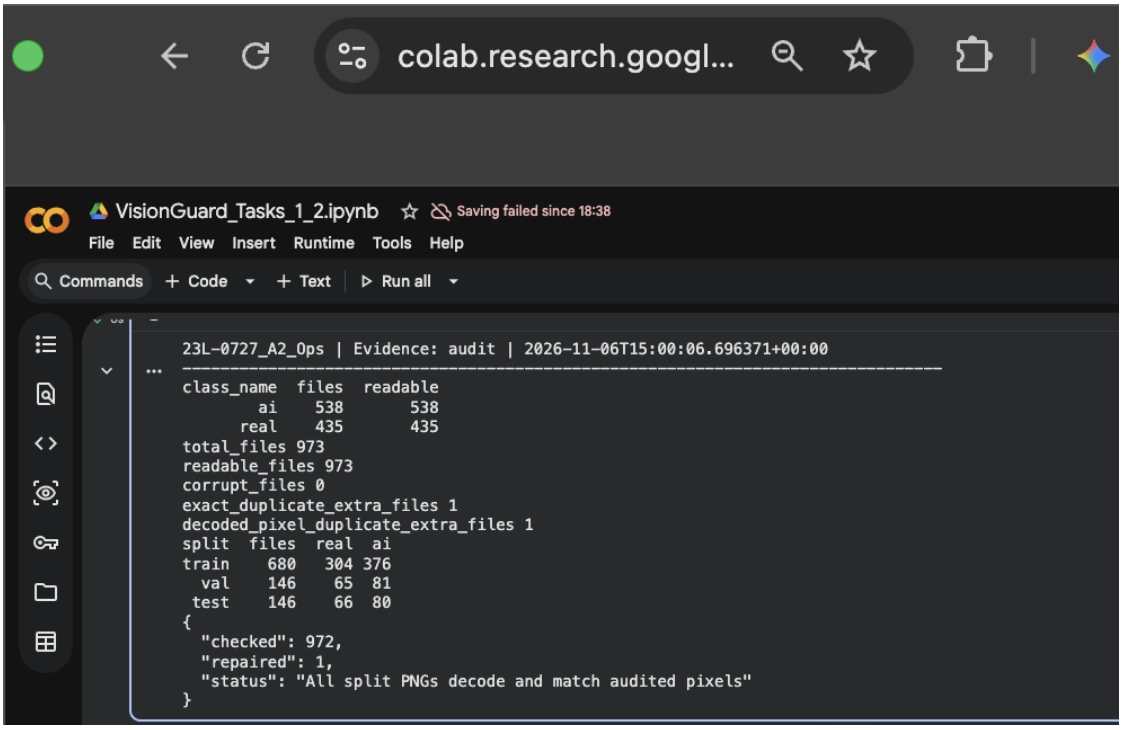


Figure 2. Executed audit and split evidence for the assignment dataset.

1.3 - Visual EDA: sample grid and widths

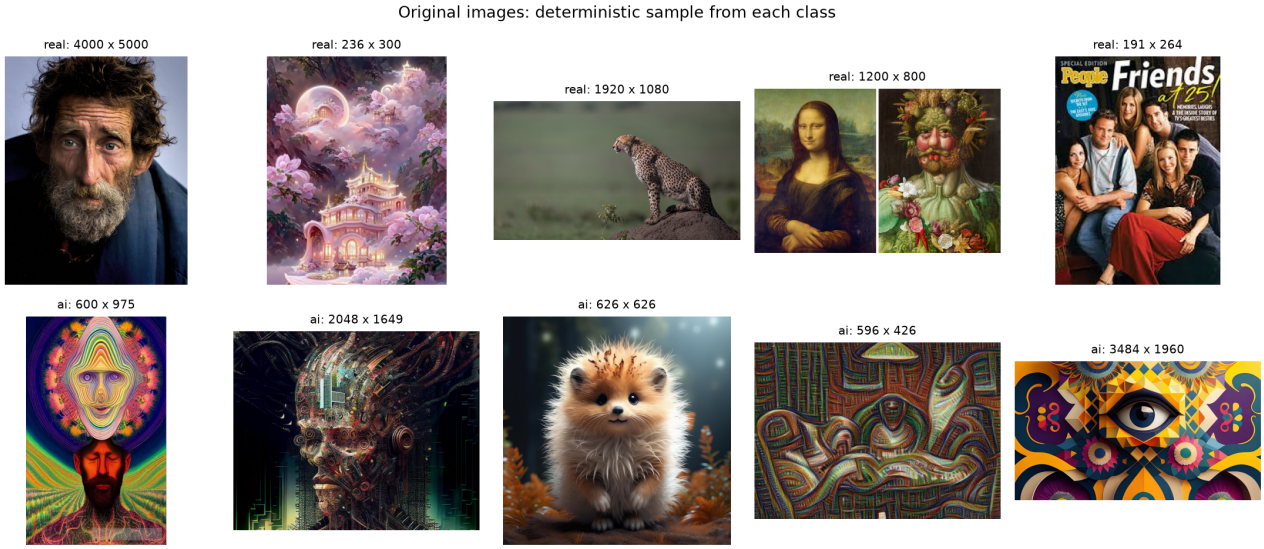


Figure 3. Deterministic original-image samples from each class (seed 42).

The real-labeled sample includes a portrait, a cheetah photograph, paintings, a magazine cover, and fantasy artwork; the AI sample includes stylized faces, abstract patterns, and a fluffy animal. This visible overlap means the problem is not simply photograph versus illustration, and the fantasy artwork in the real folder raises a label-provenance concern without proving mislabeling. Folder labels are retained as instructed; a small sample cannot establish authenticity or representativeness, and a separate label review would be needed.

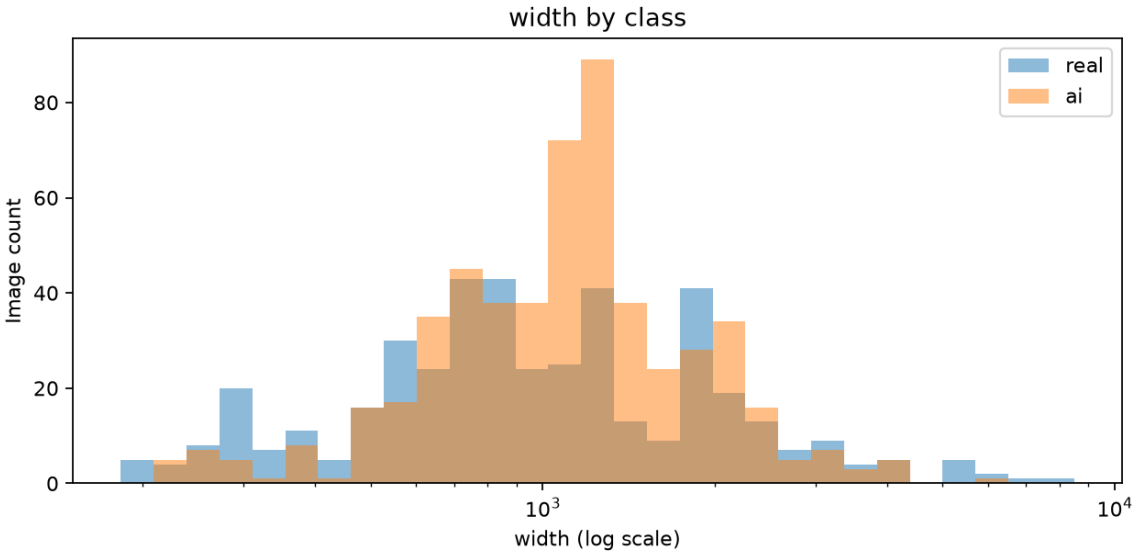


Figure 4. Original image width distribution by class; horizontal axis is logarithmic.

Median width is 1028.5 pixels for AI images and 940 pixels for real images, with substantial overlap. Real images have the more extreme maximum (8495 versus 6000 pixels). Resolution is therefore a possible source shortcut and compute cost; resizing makes tensor shapes consistent but does not establish that the detector has learned generation artifacts.

1.3 - Visual EDA: heights and file sizes

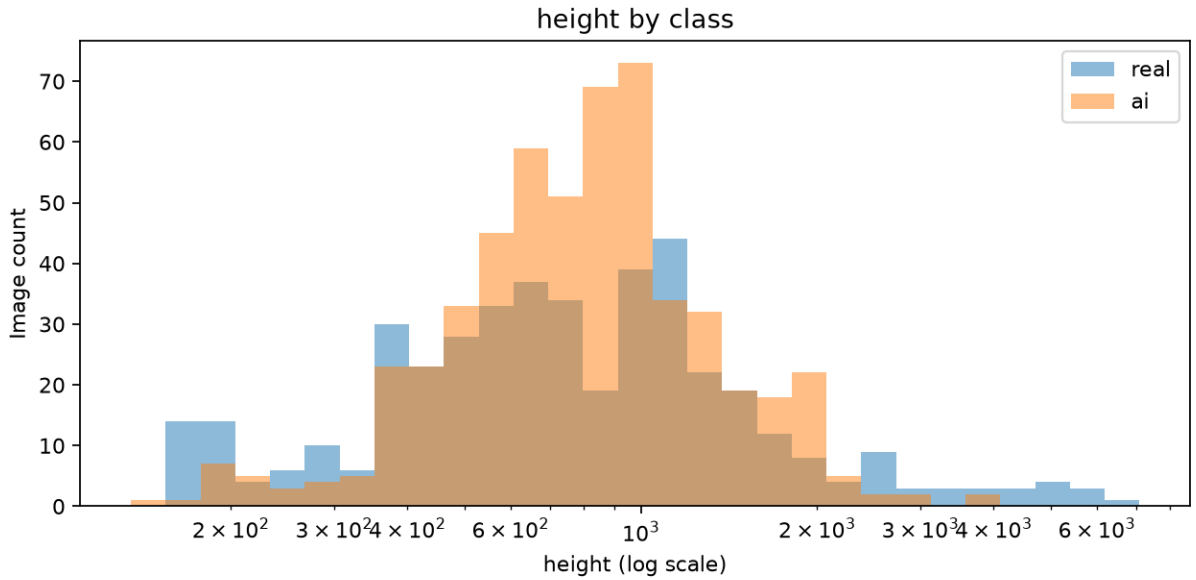


Figure 5. Original image heights by class on a logarithmic axis.

Median height is 800 pixels for AI images and 720 for real images. Both distributions span small and large images, and real images reach 7091 pixels compared with 4001 for AI. A fixed 224-pixel input reduces memory use, while potentially discarding fine forensic details.

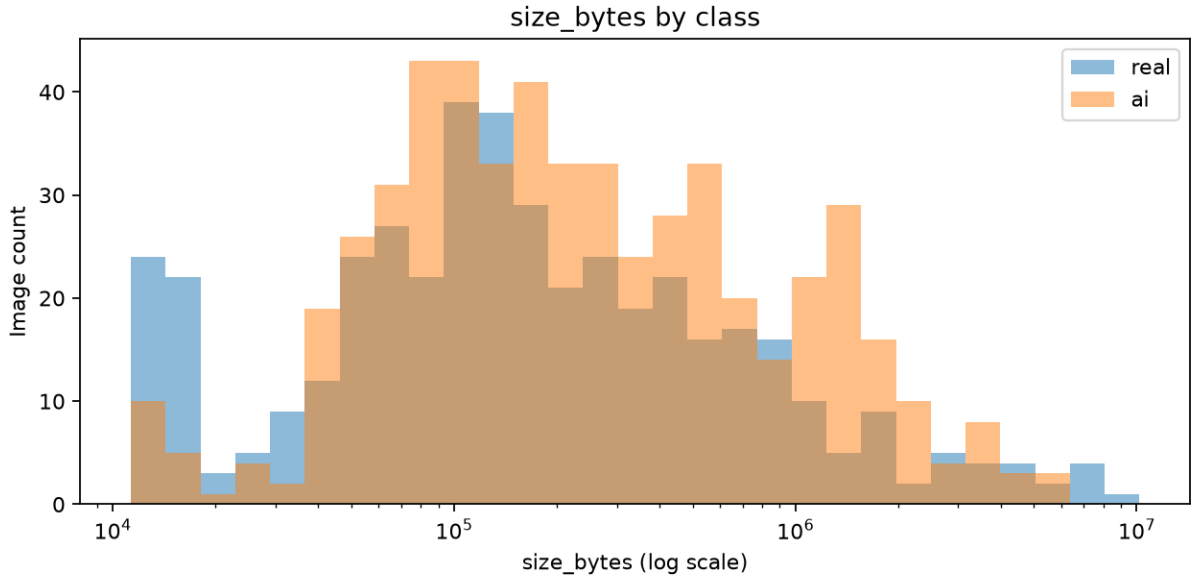


Figure 6. Original file-size distributions by class on a logarithmic axis.

The AI median file size is about 199.8 kB, compared with 145.0 kB for real images, although the largest real file is about 10.2 MB. These are original compressed file sizes, not processed PNG sizes or tensor memory. Compression format and source quality can correlate with class, so file size alone would be an unreliable authenticity signal.

1.4 - Git and DVC: toy round trip

A new empty directory was initialized with Git and DVC. tiny.csv initially contained two labeled rows; dvc add created its pointer, and Git committed the metadata. A third row was added and committed as a second version. Both data versions were pushed to a configured local-folder DVC remote, which the assignment permits.

Toy evidence	Observed value
Version 1 commit	edae2050b580c170042a20a80adb59d536f783b5
Version 2 commit	911bc3f41a0ea52a11ea3490bd8f8f18f3f653fa
Restored version	edae2050b580c170042a20a80adb59d536f783b5
Recovery assertion	True

git checkout selected the first commit and dvc checkout restored its two-row content. The toy CSV and its local DVC cache were then removed. dvc pull recovered the older bytes from the remote, and the script asserted exact content equality. Deleting the cache makes this stronger than recovering from a local cached copy.

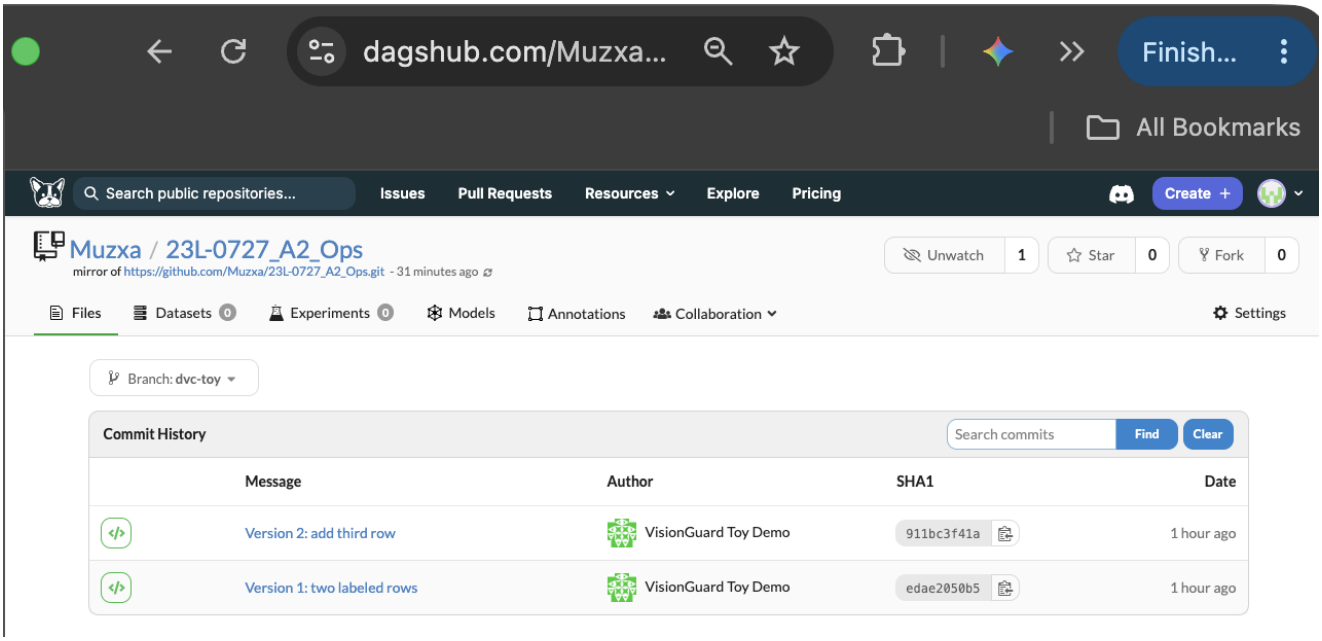


Figure 7. Both toy Git commits published on the DagsHub dvc-toy branch. Recovery is verified in the accompanying transcript and result.json.

git init starts metadata history; dvc init creates DVC configuration; dvc add hashes data and writes pointers; dvc remote add chooses object storage; dvc push uploads objects; dvc pull retrieves the objects named by the checked-out pointer. The two toy commits are also published to the DagsHub project as described in the accompanying repository evidence.

1.4 - Apply DVC to the real dataset

The original dataset, normalized RGB copies, and split manifests were tracked separately with DVC. dvc push uploaded their objects to the persistent local remote under versioned_storage/dvc_remote. Git stores the small pointers and configuration; a bare Git backup preserves metadata independently of the working tree.

Actual raw pointer:
outs:
- md5: b2cec67839f742df4653f66d4bcbbafb.dir
size: 503783378
nfiles: 975
hash: md5
path: raw

The pointer contains the content checksum, byte size, file count, hash algorithm, and path. For a directory, its .dir object identifies the file manifest. The audit uses SHA-256 for source identity; DVC's content checksum serves its own storage protocol. Large images remain outside Git because binary versions inflate history and do not produce useful text diffs.

The raw DVC pointer counts 975 files, while the image audit counts 973 candidates: DVC also includes two hidden .DS_Store files ignored by the image audit. This difference does not indicate two unreadable images.

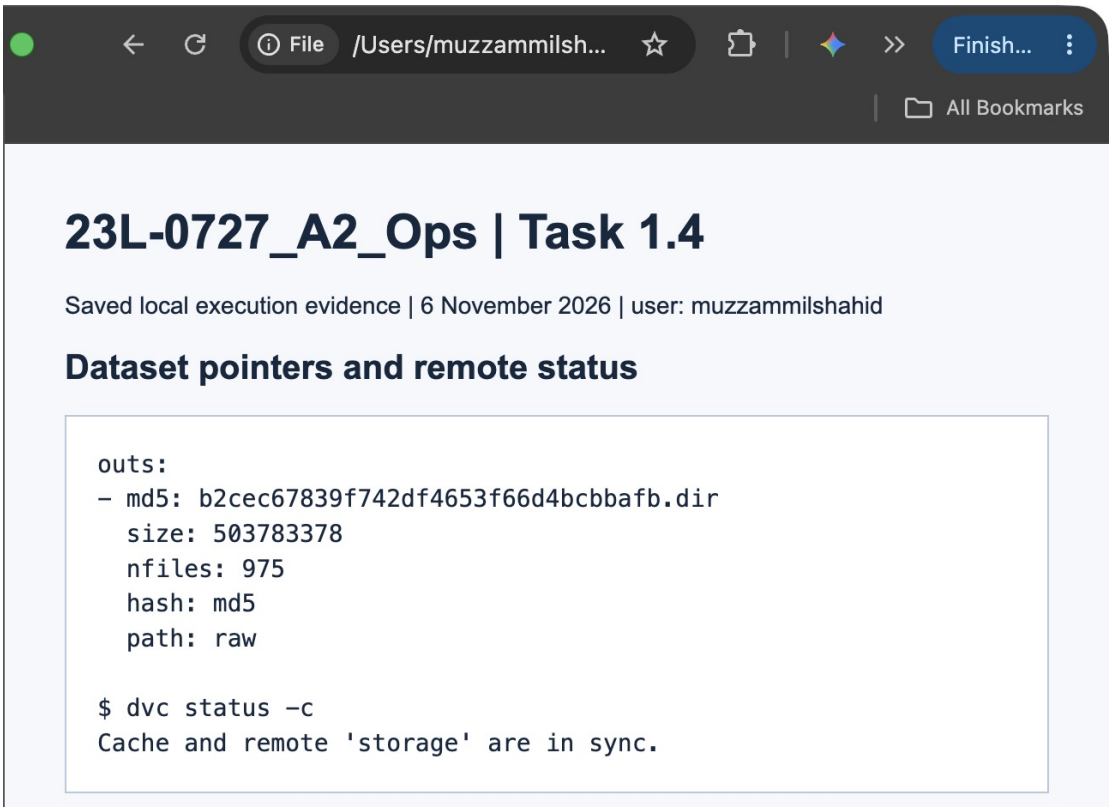


Figure 8. Browser view of the actual saved raw pointer and freshly executed DVC remote status. The log file is included in the evidence bundle.

1.4 - Published project and version history

Screenshots are cropped browser captures that preserve the displayed evidence and project context while excluding the software cursor overlay. No reported values were edited. The GitHub project is mirrored to DagsHub under the required roll-number project name. The dvc-toy branch preserves both CSV versions, while main contains the audited dataset pointers and model development source. Figure 9 shows the published project; large data objects remain in the permitted local DVC remote.

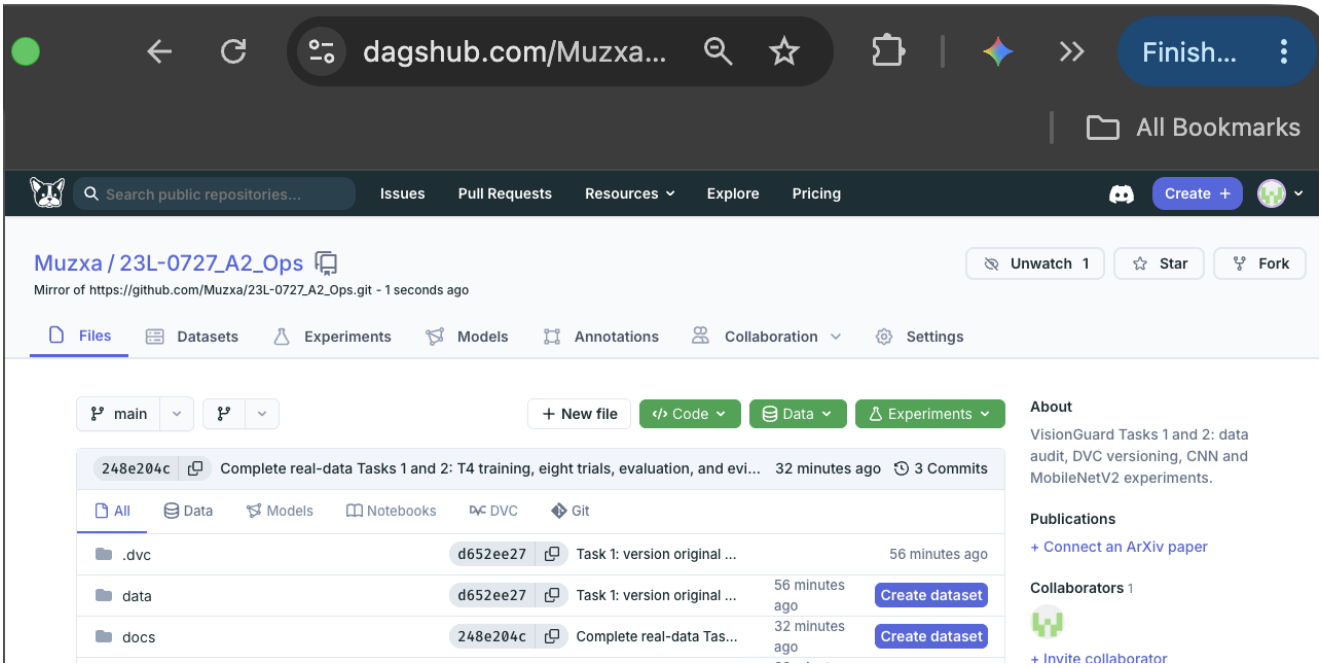


Figure 9. DagsHub project and source version evidence.

The local remote satisfies the permitted storage option. Reproducing on another computer requires access to that remote or an exported copy and an updated remote path; a Git clone alone cannot reconstruct excluded image bytes.

Task 2 - Model Development with TensorFlow/Keras

2.1 - Input pipeline and augmentation

Saved CSV split lists supply file paths and float labels. tf.data reads each RGB PNG, decodes three channels, resizes with bilinear interpolation and antialiasing, and pads to 224 x 224 without stretching. This matches MobileNetV2's standard input and limits GPU memory, at the cost of losing small artifacts. Padding can itself become a shortcut if aspect ratios correlate with labels.

The deterministic resized tensors are cached in memory before training shuffle. Training reshuffles each epoch with seed 42; batches contain up to 32 examples, and one batch is prefetched. Decode parallelism is capped at two to limit memory. Cache avoids repeatedly decoding large PNGs, and does not cache augmentation. Validation/test preserve manifest order and use no augmentation.

Training-only augmentation lives inside each model: horizontal flips, rotation up to 0.025 turns, zoom factor 0.08, and contrast factor 0.08. Reflect padding reduces blank fill during geometric augmentation. Keras disables these random transforms during evaluation; saved preprocessing therefore travels with the model.

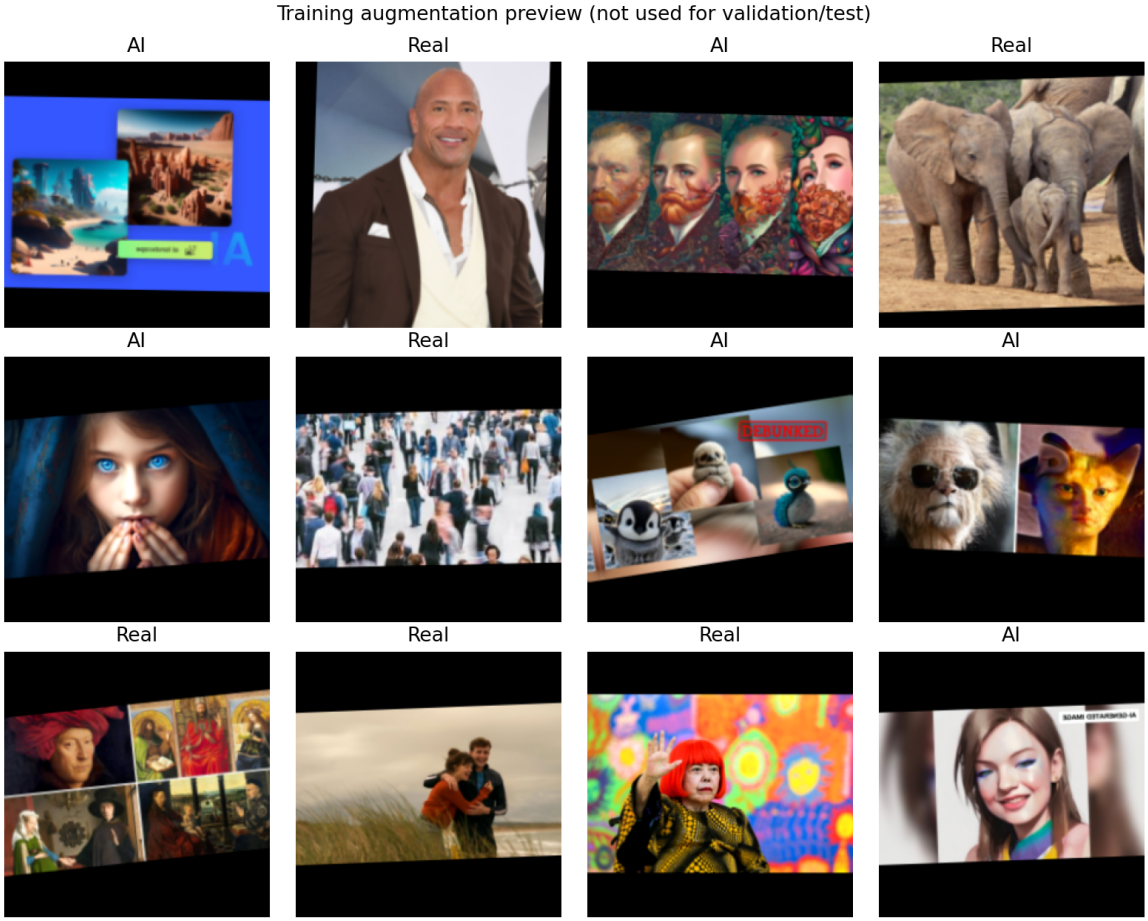


Figure 10. A genuine augmented training batch. Labels come from the manifest.

Observed repair. One processed PNG failed TensorFlow decoding. Its source byte hash and decoded pixel hash were checked against the original audit, then its RGB pixels were re-encoded with TensorFlow. All 972 split images passed decoding and pixel-hash verification; no examples or labels were dropped.

2.2 - Scratch CNN architecture

The scratch network has three convolution blocks (32, 64, 128 filters). Each 3 x 3 convolution shares local feature weights; BatchNorm stabilizes activations, ReLU adds nonlinearity, and max pooling reduces spatial size and computation. Global average pooling avoids a very large flattened classifier. A 64-unit ReLU layer combines channels; dropout 0.4 regularizes, and one sigmoid produces an AI score.

Layer	Output shape	Parameters
rgb_pixels_0_255	(None, 224, 224, 3)	0
train_only_augmentation	(None, 224, 224, 3)	0
scale_0_1	(None, 224, 224, 3)	0
conv_1	(None, 224, 224, 32)	864
bn_1	(None, 224, 224, 32)	128
relu_1	(None, 224, 224, 32)	0
pool_1	(None, 112, 112, 32)	0
conv_2	(None, 112, 112, 64)	18,432
bn_2	(None, 112, 112, 64)	256
relu_2	(None, 112, 112, 64)	0
pool_2	(None, 56, 56, 64)	0
conv_3	(None, 56, 56, 128)	73,728
bn_3	(None, 56, 56, 128)	512
relu_3	(None, 56, 56, 128)	0
pool_3	(None, 28, 28, 128)	0
global_average_pooling2d	(None, 128)	0
dense	(None, 64)	8,256
dropout	(None, 64)	0
p_ai	(None, 1)	65

The loss is binary cross entropy because the target is binary and the output is a sigmoid. Adam starts at 0.001, providing adaptive updates. Training monitors binary accuracy, AUC, precision, and recall; the latter classification metrics use 0.5 during fit. Final metrics instead use thresholds chosen solely from validation. Sigmoid scores are not guaranteed calibrated authenticity probabilities.

2.2 - Model summary evidence

Figure 11 shows the actual layer evidence exported from the trained scratch model. Shapes follow the data flow in the preceding table: three pooling operations reduce 224 x 224 to 28 x 28, and global average pooling reduces each feature map to one channel value. The screenshot focuses on layers with parameters. Convolutional layers dominate this small network; augmentation and pooling have no learned parameters.

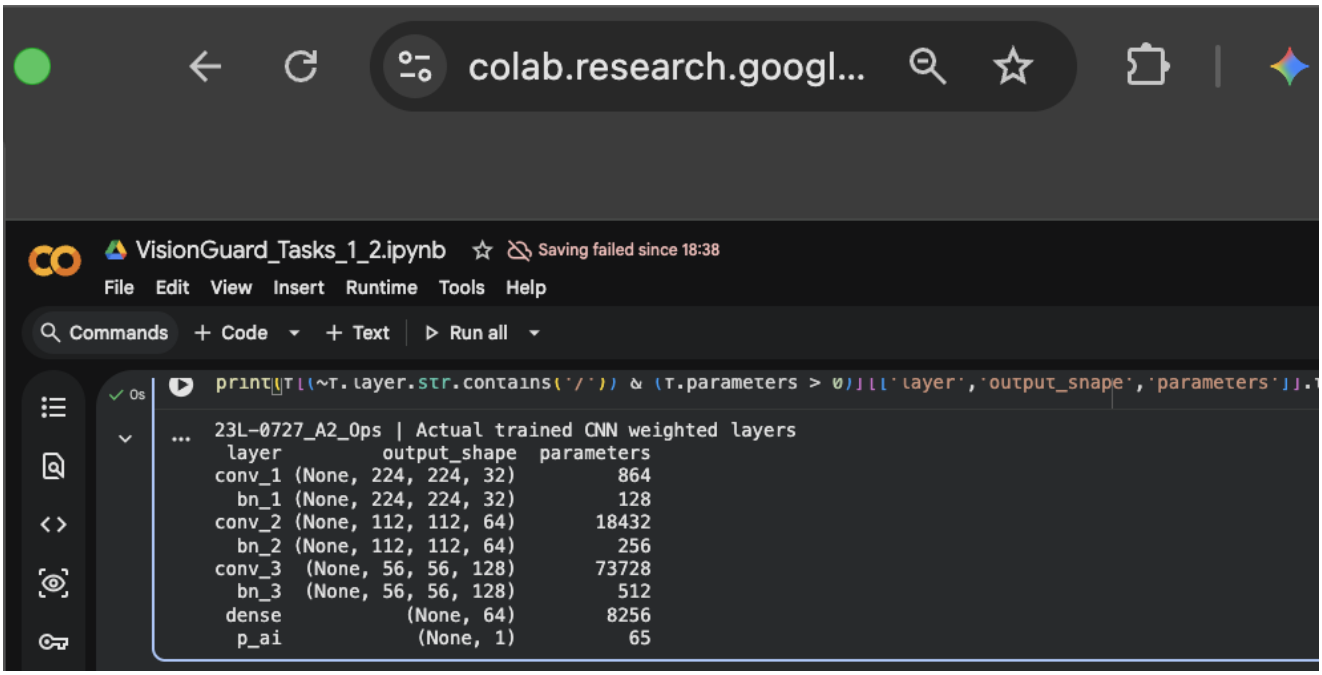


Figure 11. Executed weighted-layer excerpt of the scratch CNN, with output shapes and parameter counts.

2.3 - Transfer learning in two stages

MobileNetV2 uses ImageNet pretrained weights with include_top=False. The added classifier consists of global average pooling, dropout 0.3, and one sigmoid. Embedded rescaling maps 0-255 pixels to [-1,1], matching MobileNetV2 preprocessing. ImageNet features provide a useful initialization, while the authenticity task differs from the original object classification task.

Stage 1 freezes the backbone and trains the head with Adam at 0.001. Stage 2 unfreezes only the last block's weighted non-BatchNorm layers and the terminal convolution, then recompiles with learning rate 0.00001 (100 times smaller). BatchNorm remains frozen and the backbone is called in inference mode to preserve its moving statistics.

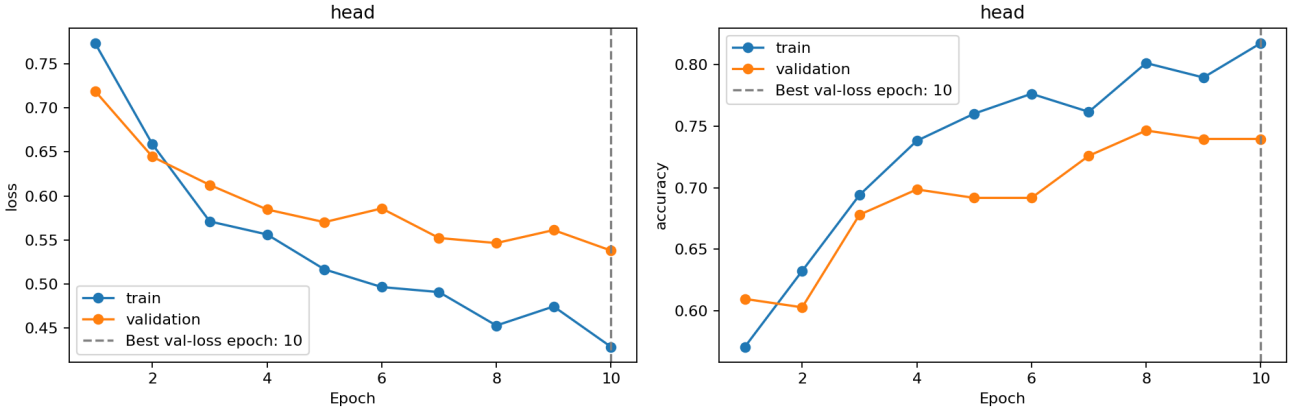


Figure 12. head: loss and accuracy curves, with minimum validation-loss epoch marked.

head trainable weighted layers: p_ai

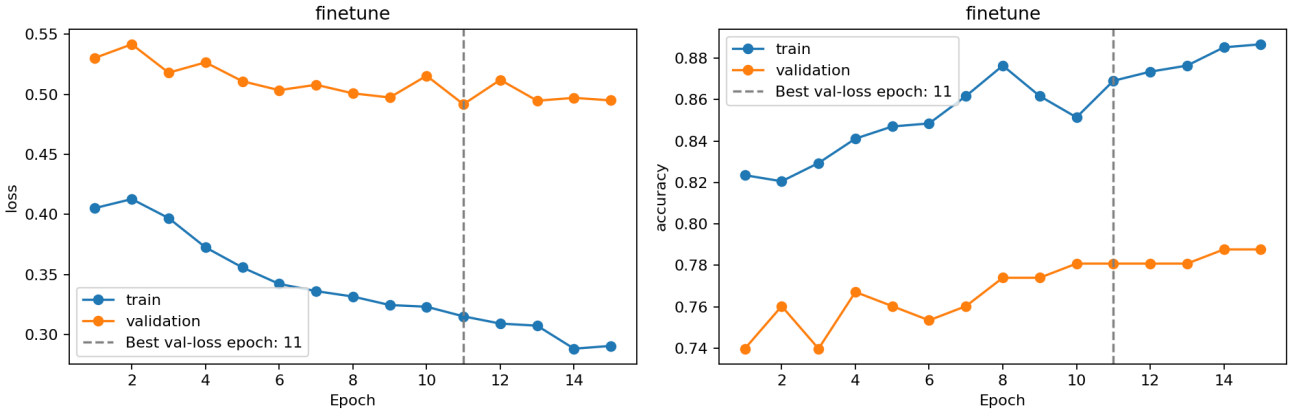


Figure 13. finetune: loss and accuracy curves, with minimum validation-loss epoch marked.

finetune trainable weighted layers: mobilenetv2_1.00_224/block_16_expand, mobilenetv2_1.00_224/block_16_depthwise, mobilenetv2_1.00_224/block_16_project, mobilenetv2_1.00_224/Conv_1, p_ai

The best validation loss fell from 0.53795 for the frozen head to 0.49179 after last-block adaptation. This supports useful adaptation on this validation split, while the later tuner winner achieved 0.47383. The frozen-head best epoch was the budget limit (10), so early stopping did not establish convergence for that stage.

2.4 - Training discipline and checkpoints

Python, NumPy, and TensorFlow seeds are fixed to 42, and TensorFlow deterministic operations are enabled. Exact reproducibility still depends on hardware, CUDA, and package versions. A checkpoint is saved only when validation loss improves. Early stopping uses patience 4 and restores best weights; ReduceLROnPlateau halves the learning rate after 2 stagnant epochs with minimum 1e-7.

Stage	Epoch budget	Epochs run	Best epoch	Best validation loss
baseline	20	10	6	0.66881
head	10	10	10	0.53795
finetune	15	15	11	0.49179

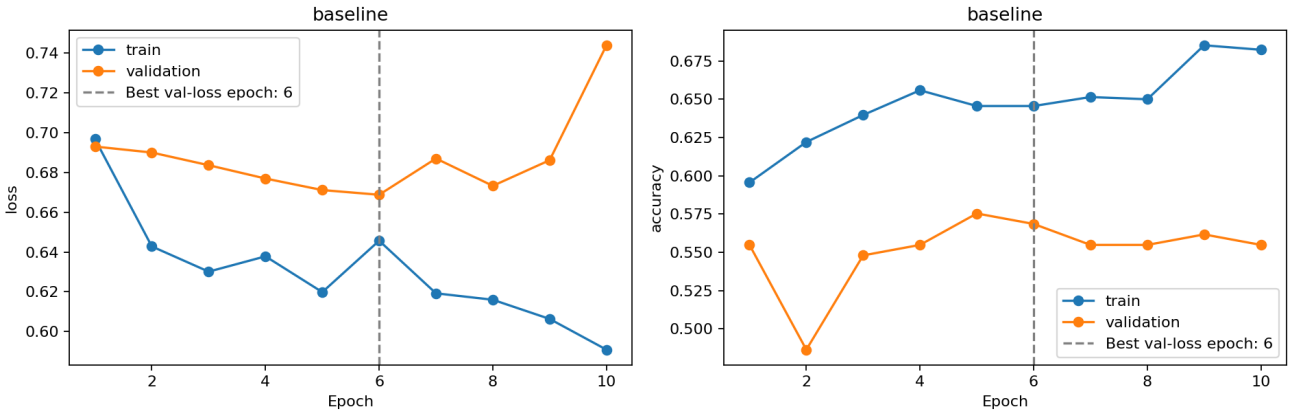


Figure 14. Scratch CNN training curves, with the selected validation-loss checkpoint marked.

Scratch training loss generally decreases, while validation loss improves only to epoch 6 and worsens later. Training accuracy rises above validation accuracy, which stays near the class-majority level. This gap is consistent with overfitting or dataset mismatch; the curves alone cannot identify its cause.

The transfer candidates (frozen head, last block, and tuner winner) were compared by validation loss before test access. The selected candidate was **tuned**. Thresholds and split hashes were saved in selection_before_test.json, then an exclusive test-start marker prevented a second final evaluation in this run directory.

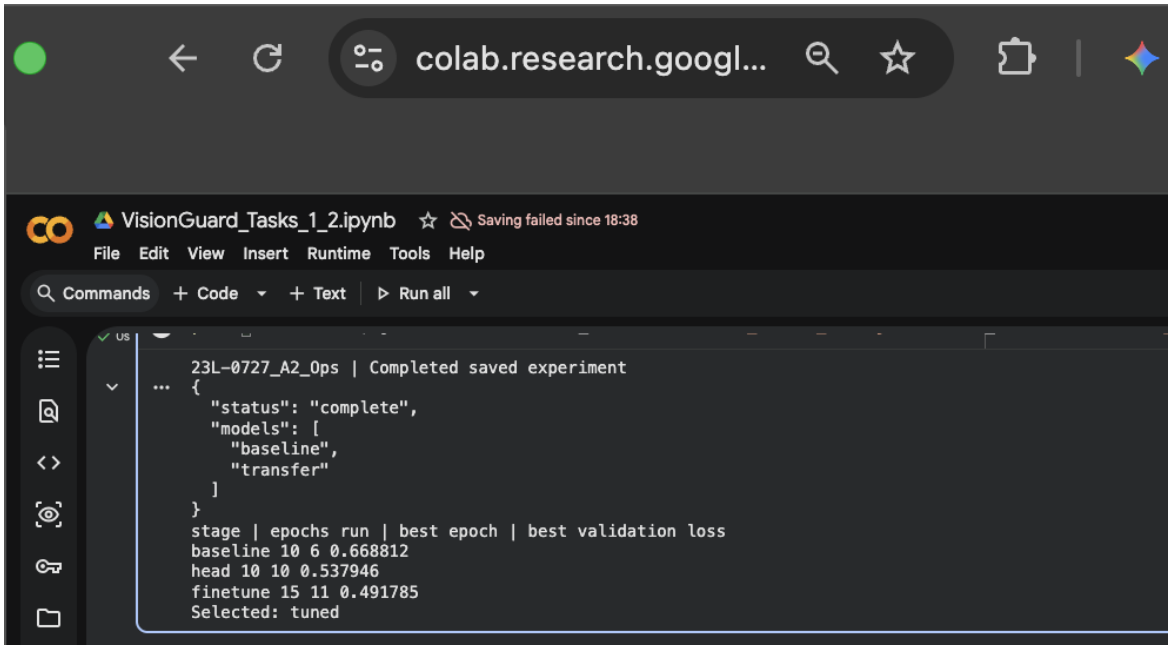


Figure 15. Actual completed experiment evidence and stage results.

2.5 - Held-out evaluation and decision threshold

AI is the positive class. A real photograph incorrectly called fake can damage trust, so the predeclared validation objective is 2 x false positives + false negatives. This 2:1 ratio is a product judgment, not a measured business cost. The algorithm checks every attainable validation boundary and chooses the higher threshold when costs tie. Test labels never influence this selection.

Model	Threshold	Accuracy	Precision	Recall	F1	ROC-AUC
baseline	0.65103	0.4932	0.6667	0.1500	0.2449	0.6672
transfer	0.68069	0.7055	0.7761	0.6500	0.7075	0.8377

Model	True real (TN)	Real called AI (FP)	AI missed (FN)	True AI (TP)
baseline	60	6	68	12
transfer	51	15	28	52

baseline: 6 of 66 real images were called AI, and 68 of 80 AI images were missed. Precision measures correctness of AI accusations; recall measures the fraction of AI images caught. ROC-AUC uses continuous scores and does not depend on the selected threshold.

transfer: 15 of 66 real images were called AI, and 28 of 80 AI images were missed. Precision measures correctness of AI accusations; recall measures the fraction of AI images caught. ROC-AUC uses continuous scores and does not depend on the selected threshold.

The tuned transfer model is the preferred model from pre-test validation loss (0.47383 versus 0.49179 for last-block fine-tuning and 0.53795 for the frozen head). The held-out comparison also favors transfer: F1 is 0.7075 versus 0.2449 and ROC-AUC is 0.8377 versus 0.6672. However, it makes more false accusations (15 versus 6), so this preference is not a claim that every error type improves. The scratch threshold is conservative and misses 68 of 80 AI images; its training/validation gap and weak validation discrimination limit its usefulness. Test weighted cost is 58 for transfer versus 80 for scratch under the declared 2:1 rule; these are retrospective comparisons, and no further tuning followed them.

The 146-image test set is small and comes from the same collection as training. These are point estimates without a new-source or new-generator guarantee. Source artifacts, label uncertainty, compression, and unseen generators limit generalization; the detector should support review rather than establish authenticity.

2.5 - Executed metric evidence

Figure 16 records the final saved test results in the actual Colab notebook. The report tables use the unrounded JSON values, whereas this compact display rounds to four decimal places. Confusion-matrix counts preserve the exact errors behind precision and recall; all values refer to the same 146 held-out images.

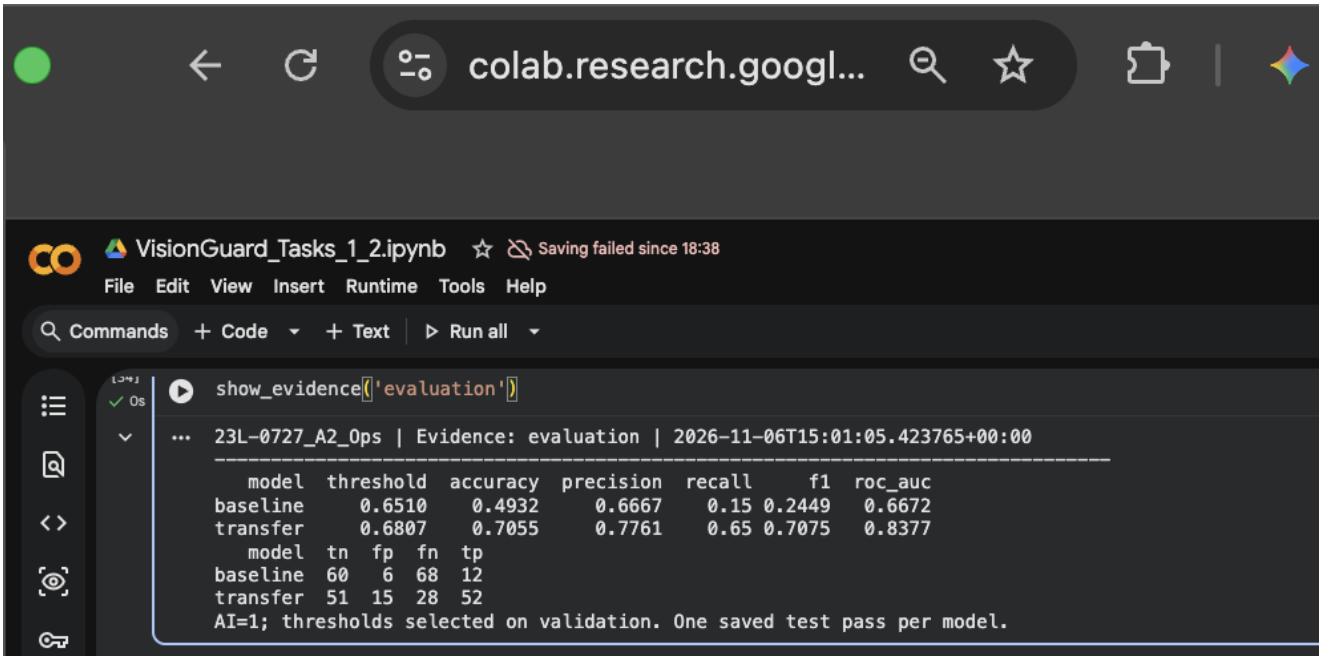
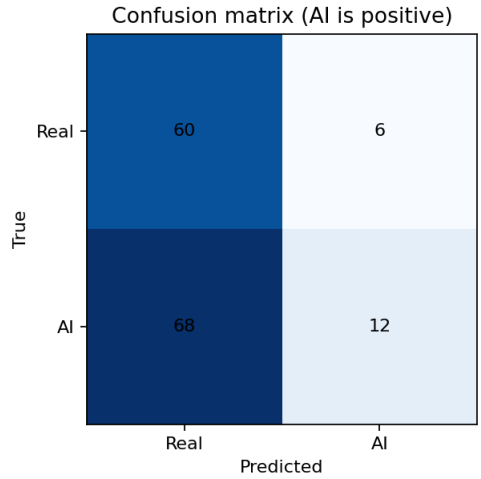
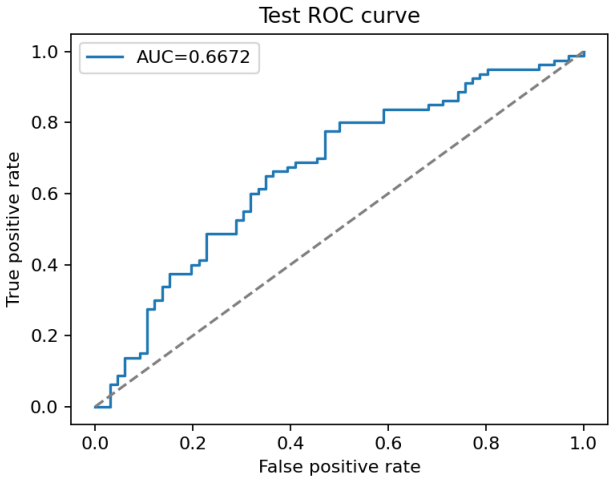


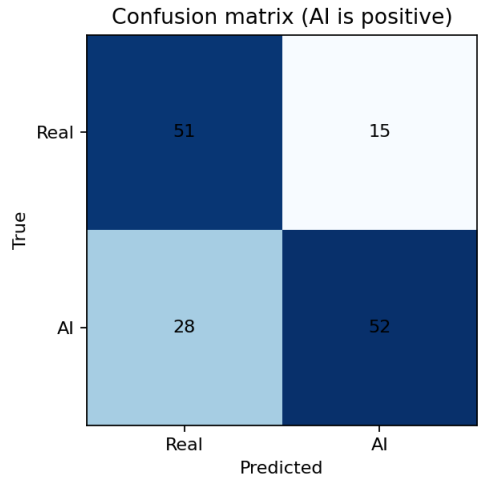
Figure 16. Executed test metric comparison with the project context.

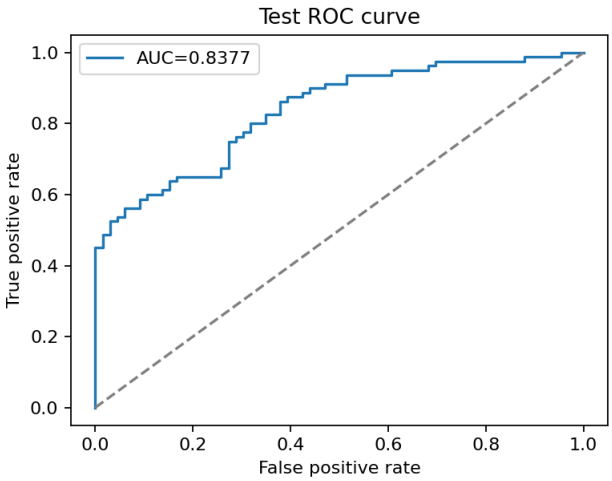
2.5 - Confusion matrices and ROC curves





baseline: confusion matrix (rows=true, columns=prediction) and held-out ROC curve.





transfer: confusion matrix (rows=true, columns=prediction) and held-out ROC curve.

Each final model predicts the test set exactly once. Probabilities, row-level predictions, and labels are saved, and the confusion matrix, ROC, metrics, and error grids are derived from that same pass. Accuracy is (TP+TN)/N; precision=TP/(TP+FP); recall=TP/(TP+FN); F1 is their harmonic mean. No model was retrained after the held-out metrics were exposed.

2.6 - Error analysis: transfer

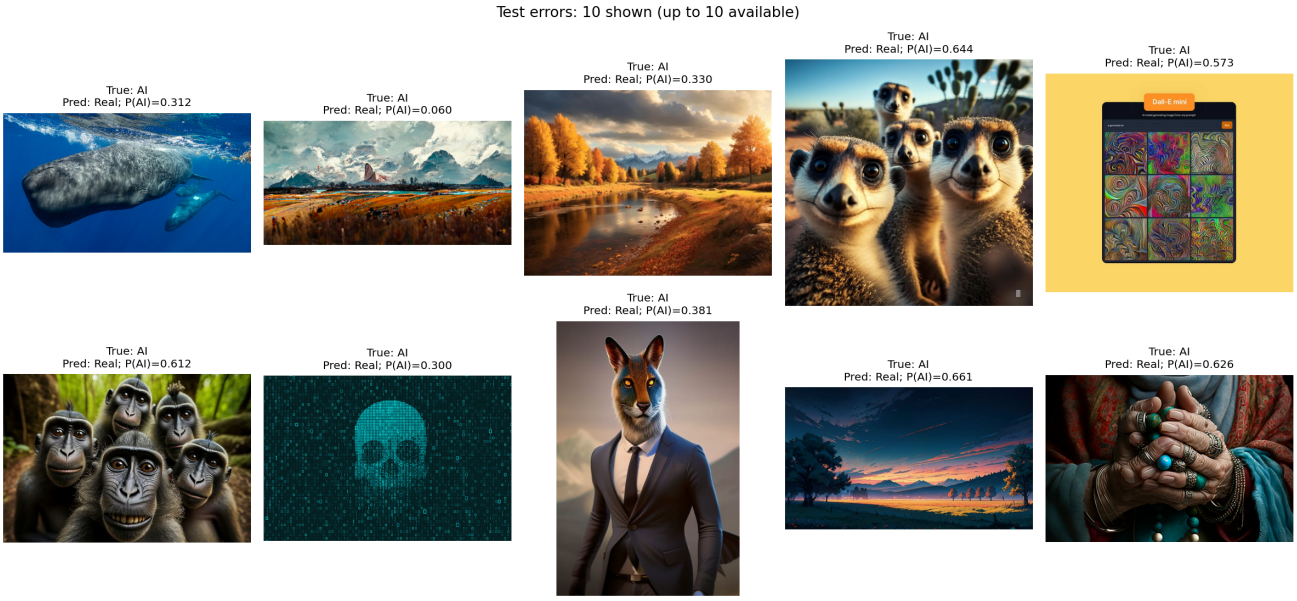


Figure 17. First ten available transfer errors in manifest order; true and predicted labels and AI scores are shown.

Image	Visible observation	Hypothesis and check
1	A whale underwater has plausible lighting and water texture.	ImageNet wildlife features may favor a real-like score; test matched unseen synthetic/real underwater images.
2	A broad colorful meadow is framed by snow-covered mountains.	Natural scene semantics may hide generation artifacts at 224 px; check a separate high-resolution landscape set.
3	Orange autumn trees and a reflective river fill a landscape.	Texture and lighting resemble photography; compare independent real/generated autumn scenes with controlled compression.
4	The foreground animals have large eyes and close-up fur texture.	Recognizable animal fur may dominate unusual proportions; compare unseen stylized renders with real animal close-ups.
5	A yellow page surrounds a Dall-E mini interface and a nine-image grid.	The surrounding screenshot may dilute the small generated tiles; compare tile crops with full screenshots on new examples.
6	Five primate-like faces cluster in front of a green background.	Familiar faces may outweigh stylized facial geometry; test independent generated/real primate portraits.
7	A turquoise skull is drawn using small binary digits on a dark field.	Graphic art also occurs in the real folder; inspect label provenance and evaluate independently curated digital artwork.
8	An upright animal-headed figure wears a suit and tie.	Real-folder illustrations may overlap anthropomorphic art; compare independent composite/illustration categories.
9	A stylized sunset landscape uses broad dark-blue and orange color bands.	Painting-like real data may share this visual style; test a separately sourced illustration holdout.
10	A close-up shows many fingers, rings, beads, and patterned fabrics.	Local hand geometry may be missed by global pooling; inspect high-resolution hand regions on new examples.

Images are numbered left-to-right, top-to-bottom. Sorted paths place AI images first, so these first ten available errors are all false negatives; the confusion matrices cover both error types. These are visual observations and testable hypotheses, not proven causes. Proposed diagnostics were not used to retune the model or threshold.

2.7 - Keras Tuner: Fashion-MNIST toy

The toy uses a tiny flattened Fashion-MNIST classifier with one hidden ReLU layer and a 10-class softmax. A fixed stratified subset of 10,000 training and 2,000 validation images is taken from the built-in training partition; its test partition remains unused. Pixels are scaled to [0,1], with sparse categorical cross entropy for integer class labels.

Random search explores 32/64/128 hidden units and Adam learning rates 0.01/0.001/0.0001. Eight trials run up to five epochs with early stopping (patience 2). The objective is minimum validation loss, so lower is better.

Trial	Status	Units	Learning rate	Validation loss
1	COMPLETED	128	0.001	0.45032
4	COMPLETED	64	0.001	0.45060
3	COMPLETED	32	0.001	0.45396
0	COMPLETED	64	0.01	0.48048
5	COMPLETED	32	0.01	0.48549
2	COMPLETED	128	0.01	0.49178
6	COMPLETED	128	0.0001	0.52975
7	COMPLETED	64	0.0001	0.57617

Best toy settings: **{'units': 128, 'learning_rate': 0.001}**. This lab demonstrates tuning mechanics on an unrelated built-in dataset; its score is not VisionGuard performance.

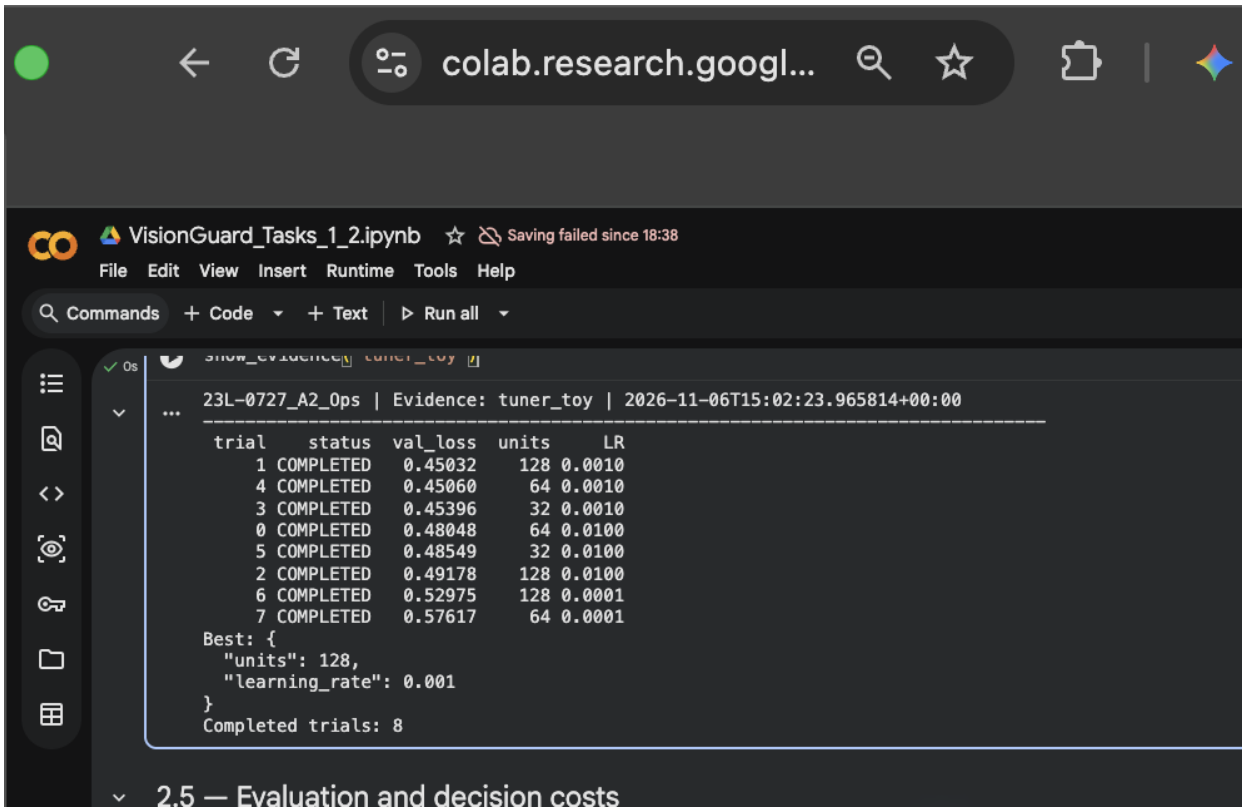


Figure 18. Actual Fashion-MNIST toy search results.

2.7 - Applied VisionGuard tuning

Every trial starts from the same trained frozen-head checkpoint. Random search varies learning rate {1e-5, 3e-5, 1e-4}, head dropout {0.2, 0.4, 0.5}, and {1, 2, 4} unfrozen weighted layers within the final MobileNetV2 block/terminal convolution. Earlier layers and BatchNorm remain frozen. Each of eight trials runs up to five epochs with early stopping and learning-rate reduction.

Trial	Status	LR	Dropout	Unfrozen	Val loss
4	COMPLETED	0.0001	0.2	2	0.47383
6	COMPLETED	0.0001	0.2	1	0.48911
1	COMPLETED	3e-05	0.2	4	0.49624
2	COMPLETED	0.0001	0.5	1	0.50121
5	COMPLETED	3e-05	0.4	2	0.51677
3	COMPLETED	1e-05	0.2	2	0.51849
0	COMPLETED	3e-05	0.4	1	0.52119
7	COMPLETED	1e-05	0.5	4	0.52495

Best applied settings: **{'dropout': 0.2, 'unfrozen_layers': 2, 'learning_rate': 0.0001}**. The actual winning checkpoint is retained; the final transfer candidate is selected against the head and standalone fine-tune candidates using validation loss.

2.7 - Applied tuner evidence and interpretation

Figure 19 confirms that all eight applied random-search trials completed and records the winning settings. Trial IDs are zero-based, so winning trial 4 is the fifth trial. These are validation results; test data did not choose the hyperparameters.

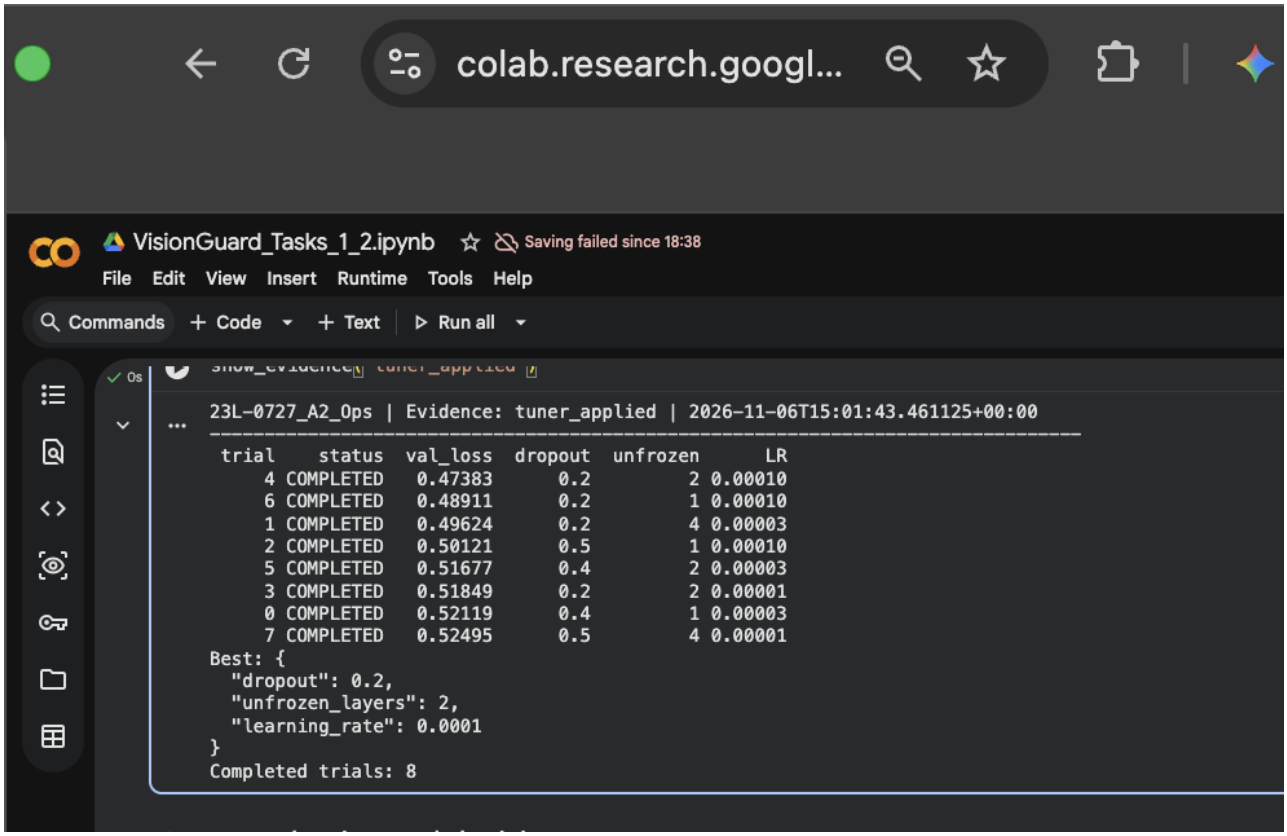


Figure 19. Eight completed applied trials and winning settings.

Parameter	Range of marginal mean validation losses
learning_rate	0.033671
dropout	0.024562
unfrozen_layers	0.007562

learning_rate has the widest observed marginal mean-loss range, and thus the strongest exploratory association in these trials. This is not causal importance: multiple settings change together, each value has few observations, and optimization is stochastic. A controlled one-factor comparison would be needed to isolate effects.

The marginal mean validation loss was 0.52172 at 1e-5 (2 trials), 0.51140 at 3e-5 (3), and 0.48805 at 1e-4 (3). Larger updates may have adapted the pretrained final layers faster within the short five-epoch budget. This is a hypothesis about the observed association, not proof that 1e-4 is universally best.

Choosing the minimum of many validation scores can select random favorable noise and overfit the validation split. The test set is consequently opened only after candidate and threshold selection. Further model changes after this report would require a fresh held-out set for an unbiased final check.

Reproduction, troubleshooting, and references

The source repository contains the audit, pipeline/models, tuning, threshold/evaluation code, and standalone notebook. The evidence bundle includes the exact environment, split manifests, source hashes, curves, checkpoints, tuner state, and saved predictions. The report builder reads these outputs and refuses to build without COMPLETE.json and at least eight completed applied trials.

Observed issue	Resolution and evidence
PNG decoding failure	Lossless re-encoding from the audited original; 972/972 pixel hashes verified, one file repaired.
Slow repeated full-resolution decode	Cache resized deterministic inputs before per-epoch shuffle; augmentation remains dynamic.
Google Drive storage quota	Use runtime-local output storage and download evidence/checkpoints. Existing Drive content was not deleted.

For a new run: create a virtual environment; install requirements; audit the originals; verify labels and split hashes; run the PNG integrity repair; execute scripts.run_training with a new output directory. Use the provided unchanged settings for a pre-test resume. After test access, read saved predictions rather than repeating model selection.

Task 3 experiment tracking and Tasks 4-8 serving, database, containers, cloud deployment, and API integration are outside this Tasks 1-2 submission. The optional Grad-CAM bonus is also outside this report. Local saved experiment evidence is not claimed as remote MLflow tracking.

Assignment dataset: https://www.kaggle.com/datasets/cashbowman/ai-generated-images-vs-real-images

TensorFlow transfer learning: https://www.tensorflow.org/guide/keras/transfer_learning

Keras Tuner getting started: https://keras.io/keras_tuner/getting_started/

DVC remote storage: https://dvc.org/doc/user-guide/data-management/remote-storage